

TEA 1 PMR

Introduction :

L'objectif de ce TEA est de créer une application Android "To-Do List" permettant de créer plusieurs listes de tâches, associées à des utilisateurs.

C'est une première prise en main d'Android Studio et Kotlin, permettant de découvrir la gestion des vues, des données, des paramètres d'application, ainsi que de l'affichage des listes.

Analyse :

1) Stockage des données

Il existe plusieurs façons de stocker de l'information pour une application Android. L'une de ces manières est d'utiliser des *SharedPreferences*¹. Ces préférences peuvent être spécifiques à l'application, ou bien être partagé entre plusieurs applications, pour, par exemple, permettre de connecter automatiquement un utilisateur sur toutes les applications d'une entreprise.

Cela a été utilisé pour le stockage du nom de l'utilisateur actuel au format *String*, ainsi que de la liste de tous les utilisateurs sous la forme d'une liste de *String* (pour des raisons qui seront expliquées dans la prochaine partie).

Il est également possible de stocker de l'information sous la forme de fichiers propre à l'application. Il faut alors décider du format de données utilisé. Dans notre cas de figure, nous avons stocké les informations au format JSON, à l'aide de la librairie GSON² qui nous permet de sérialiser et désérialiser directement des objets Kotlin.

Une structure de données a donc été élaboré dans *data.Todos.kt*, ainsi qu'une interface dans *data.DataProvider*, afin de fournir un niveau d'abstraction pour l'accès et la sauvegarde des données au reste de l'application.

Chaque utilisateur a son propre fichier, contenant toutes ses listes et tâches.

Note : GSON est en réalité déconseillé pour les applications Android. Cf [GSON README](#).

2) Gestion des préférences

Pour la création d'un panel de préférence, *AndroidX Preference Library*³ a été utilisé plutôt que *PreferenceActivity*, car ce dernier n'est plus supporté, et la documentation est plus compliquée à trouver.

¹ [Enregistrer des données simples avec SharedPreferences | App data and files | Android Developers](#)

² [gson/UserGuide.md at main · google/gson · GitHub](#)

³ [Settings | Views | Android Developers](#)

On peut alors facilement créer des paramètres pour les *SharedPreferences*, et des boutons ont été rajoutés pour vider les données de l'application, ainsi que générer un jeu de données de test.

L'onglet est injecté dans toutes les vues à l'aide d'un class *BaseActivity*, dont héritent toutes les autres classes.

3) Affichage des listes

Comme vue en cours, nous avons utilisé un *RecyclerView*⁴ afin de dynamiquement afficher les listes, ainsi que les tâches.

L'affichage des listes et des tâches étant similaire, une classe *BaseAdapter* et *BaseViewHolder* ont été créées, desquelles héritent les deux vues.

La gestion de la partie affichage se fait au niveau des *ViewHolder*, et la gestion des données au niveau de l'activité. Les données sont sauvegardées en mémoire à chaque mise en pause d'une vue (changement de vue, ou bien fermeture de l'application).

4) Bonus

Il est possible de renommer les listes en appuyant longtemps dessus, renommer les tâches, ainsi que supprimer les listes et les tâches.

Conclusion :

Ce TEA nous a appris à créer une application comportant plusieurs vues, plusieurs nouveaux éléments UI comme la barre d'action, ainsi que la gestion de rétention d'informations via l'utilisation de *GSON* et *AndroidX Preference*.

Cet exercice nous a aussi permis de voir les différences et ressemblances entre l'architecture d'une application et d'un site : sur un site, nous n'avons pas besoin de gérer la partie UI dont s'occupe le navigateur, et la philosophie d'UX n'est pas la même.

Perspectives :

- Il serait intéressant de stocker les données sur un serveur externe, afin de permettre la synchronisation sur plusieurs smartphones/plateformes des listes.
- L'internationalisation n'a pas été entreprise, c'est une piste d'amélioration.
- Idem pour l'adaptation de la vue en fonction de l'orientation du téléphone (bien que l'UI actuel reste tout à fait fonctionnel quelle que soit l'orientation).
- La partie décorrélation du thread UI et des tâches de fonds n'ont pas été faites. Sur une application aussi simple (et grâce à la puissance des téléphones récents) cela n'affecte pas le temps de réponse.

⁴ [Create dynamic lists with RecyclerView | Views | Android Developers](#)